Then there are a few methods you'll probably use less often:

attach_file(locator, path)
: This looks for a form-file upload element that matches the locator, and simulates attaching the file at the given path to the form.

click_button(locator)
click_link(locator)
: These are like click_on but work only for a button or a link.

uncheck(locator)
: This unchecks a check box specified by the locator.

unselect(value, from: locator)
: This method unselects the value from the select box being specified by the locator. This one is most useful for multivalue select boxes. For a single-value select box, all you need to do to deselect a value is select the new value.

The Capybara API: Querying

Capybara has a few methods designed to allow you to query the simulated browser page to check for various selector patterns in the page. This is one case where the syntax differs slightly between Minitest and RSpec.

Here the method current_url is used, which returns as a complete string the current URL that the simulated user is viewing. That's useful for testing whether your navigation links take you where they should.

The most common query method in Capybara is written as the matcher have_selector in RSpec, and as assert_selector in Minitest. The two are identical in functionality. By default, Capybara looks for a CSS selector matching the argument to the query method, using the common # shortcut for DOM ID and a dot (.) for DOM class. The assertion passes if the selector is found.

The Capybara query methods, including have_selector, take options that limit whether a selector with the given HTML tag, DOM class, and/or ID matches. These are the most commonly used:

- count. If you expect the selector to appear more than once you can specify exactly how many times, as in count: 3. You can also use minimum and maximum as options, and use a range with between: (1 .. 3)
- text. This takes a string or a regular expression and matches if the text of the selector either contains the text or matches the regular expression.